

PROTOS

PROTOS

UnrealEngine 4
In-Game, Multi-user
High Density Modular
World/Entity Construction System

HeadlessStudios.com

TechLord Frankie Taylor
DPROBOY Abde arrahman Dkour
RicardoLPC Ricardo Camara

PROTOS

1- What is PROTOS:

PROTOS is an In-Game World and Entity Construction/Customization System developed by HeadlessStudios.com for supporting real-time collaborative World Building and Assembly of Modular Entities developed with UnrealEngine 4 Blueprints. PROTOS incorporates methods from multiple Modular Construction paradigms to provide a Template Construction System to derive various in-game construction and customizers.

PROTOS provides *content construction* at various layers in the game development process. It is designed with Game Developers, Game Admin, Game Players, and Game Modders in mind.

Game-

- **Developers**
Can collaboratively Pre-Fabricate Assemblies and Entities in-game, save and then load in-Editor to build Lighting, Navmeshes, and more.
A Framework for Social Game Development
- **Admins**
Can collaboratively participate in High-level Quest Design, building Dungeons and populating them with Boss/Mob/Treasure. Collaborative Develop Character Dialog and Narrative. Multi-DM RPG Campaigns
- **Players**
Can Solo or Collaboratively (with Teams/Friends) Customize Characters, Vehicles, Weapons, Build Defense Structures.
- ***Modders**
Can add new Content with easy to import system.

PROTOS

2- Features and Goals :

- Promote
 - Modular Component-based Construction of Level Architecture/Structure and Entities.
 - Collage/Cluster Construction
 - Promote Asset Modding
- Ability to switch between the Game and PROTOS in an easy and fast way :

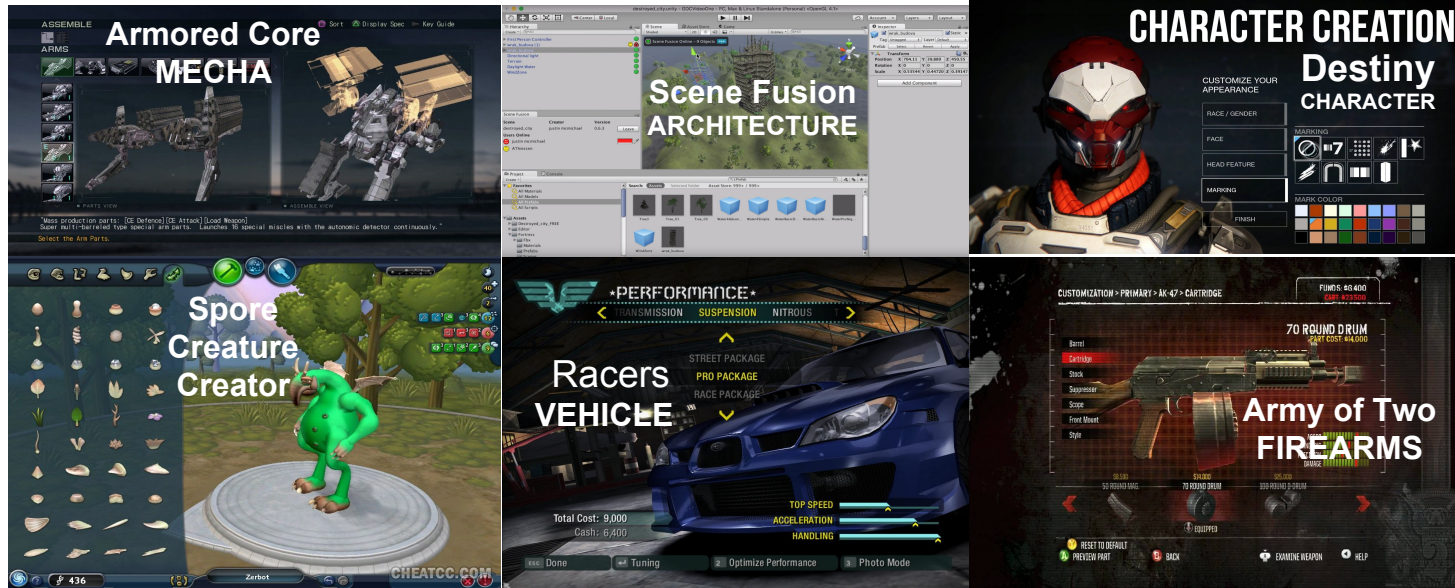
This allows for quick prototyping and testing what we add in Realtime
- Realtime Multi-User world building :

The ability to do level design and world building not only in-Game (in-Proto) but Also the ability to do it in a multi-user environment where all of the modifications are synced and for this to work , it needs :

 - Ability to access content browser for object placement
 - Ability to apply materials (Including Substance Materials)
 - Ability to Tweak the placed substance materials In-Proto .
 - Synced changes across users

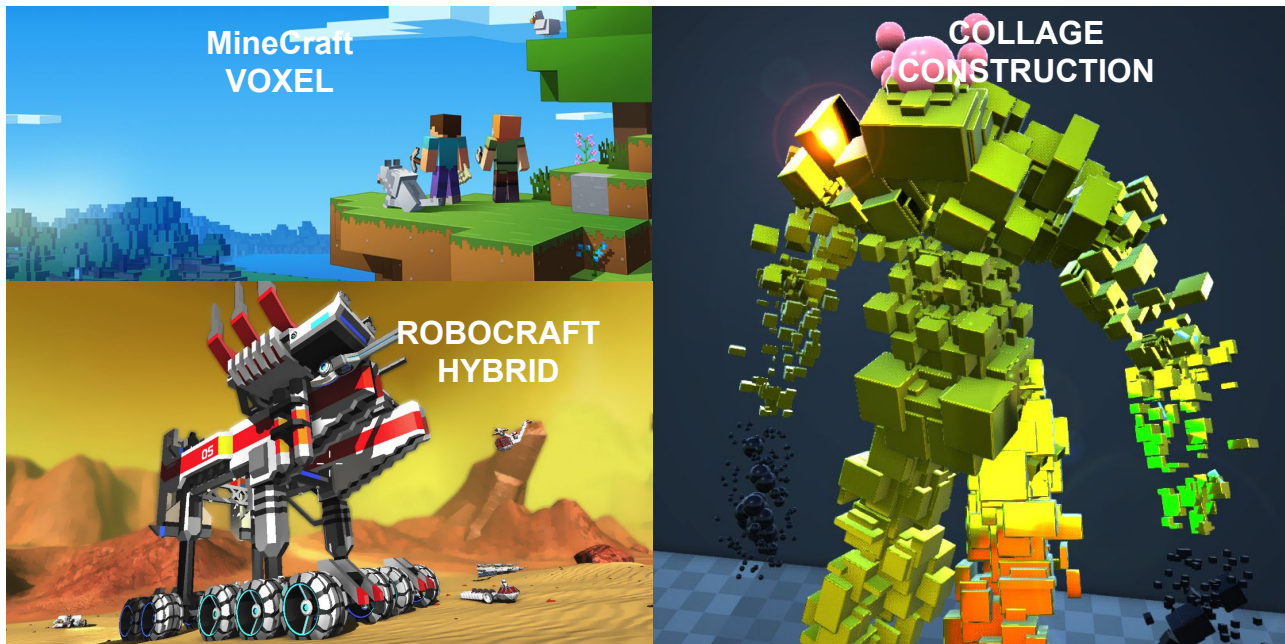
PROTOS

PROTOS is inspired by games with emphasis on creation, construction, and customization.



PROTOS

PROTOS is inspired by games with emphasis on creation, construction, and customization.



PROTOS

PROTOS is inspired by games with emphasis on creation, construction, and customization.



Entities

Organic	Mechanical	Overlays	Architectural	Items	
Characters	Projectile Weapons	Armor	Building	Ornamental Props	
Creatures	Melee Weapons	Garments Cloth Pieces	Structural Bridges/Frame work		
Tree	Vehicles (Wheeled)		Furniture Props		
	Crafts		Terrain Meshes		
	Robots				
	Machines (ie Turrets)				

PROTOS

Network Operation Modes

Developer

- Low Level Granular Assembly
- Creation of Assembly Categories
- Smart Socket Editing
- UnrealEditor Loading

Player

- In-game Customizers

Modders

- Low Level Granular Assembly

Admin

- Spawning and Orientation of Subsystem Actors
- Quest, Dialog, Player Attributes, SkillTree Creation/Editing
- Level Construction

PROTOS

Network Access:

Privilege Groups

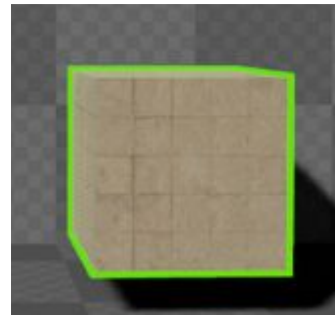
- Preset Groups: Developer, Admins, Players, Modders
- Read/Write/Execute
- View

Possession

- Checkout/Checkin
- Pass To
- Lock/Unlock
- Allow

Ownership & Identification

- Specified Color for Creator
- Post Process Object Glow of Specified Color.
- Nameplate
- Note System



PROTOS

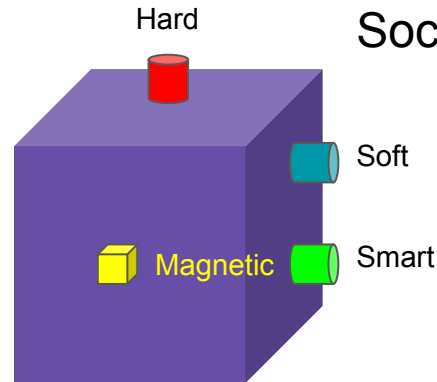
Hard, Soft, & Smart Sockets for 'Mechanical' Attachment and Assembly.

During the R &D of the Collage Construction, I devised a method of attaching mesh components in a arbitrary fashion, dubbed Soft-Sockets.

Soft Sockets are Transforms {location,rotation,scale vectors} created at run-time relative offset (local space) to the a Bones/*Hard* Sockets and stored in a save game object.

Smart Sockets are Soft sockets that are associated to the mesh asset and contain additional data & logic to filter for parts allowed/not allowed to attach. The filter data may be modified dynamically at runtime.

Some Parts use more than one Soft/Smart Socket



Sockets

Arbitrary
on-contact
attachment
To any mesh

Mesh on mesh

EDIT Modes

Free - allows attachment with no restrictions.

Soft - restricts arbitrary attachment to soft sockets.

Smart - restricts attachment based on Smart Sockets filtering.

Magnetic Mode - pulls the nearest attachable actor and automatically aligns.

Smart In place - restricts attachment based on Smart Sockets filtering switching objects in inventory via Scroll Wheel/Keyboard.

P R O T O S

Hierarchal Construction

World Actor $\Rightarrow$ Entity Actor

Basebody (DAZ3D Humanoid w/ Morphs)

Organic: Character | Creature | Plant

Mechanical Basebody: Vehicle Chassis | Building Frame | Weapon Core

Pieces/Fragment $\rightarrow$ Part $\rightarrow$ Assembly (PartsGroup) $\rightarrow$ Entity

WorldActor $\leftarrow$ Add Mesh Component

Multiple WorldActor to support Steamed
Levels/Level Occlusion

PROTOS

MVP2 Design

Goals

Support

Specified Attachment attaching certain things to specific locations

interchangeable Assemblies and Parts Swapping in-place or manual orientation

Arbitrary Object Swapping for Meshes

Arbitrary Attaching (attaching anything to anything anywhere) Various types Mesh Objects, Particles, HitBoxes, etc

swapping out entire assemblies will be interesting
example: swapping between robot arm designs that assembly could consist of upperarm - lowerarm - hand and these parts could only allow specifics part to be attach to them and those smaller parts may support arbitrary attachment to allow accents attachment

Attachment works by attaching to bone/socket

Actorcomponent to access the meshes and attach

Simplified mechanism to access and control mesh targetmorphs

Build on top of current MVP1 object control ie spawn, orientate, etc. Modify PROTOS_meshactor into a more robust object in the form of *Actor Component* that provide the functionality for any actor using it.

Add character control for to edit from various thirdperson perspective

Develop naming scheme for bones, socket, parts

Attachment Filtering Rules IE is weapon customization, where certain parts can only fit specific locations

Procedural Entity Assembly = Random Selection based on Attachment Filtering Rules

PROTOS

MVP2 PLAN Initiation

1. Create a blank actorcomponent, name PROTOS_EntityComponent
2. Create 3 Entity Actors:
 - a. PROTOS_EntityActor from Actor Class
 - b. PROTOS_EntityCharacter from Character Class (basic character or pawn with ue4man skeletonmesh)
 - c. PROTOS_EntityVehicle from Advanced Vehicle Class
3. Add a PROTOS_EntityComponent to each Entity.
4. Place Each Entity PROTOS Testmap2 to test in Multiplayer
5. Add some small parts to the Protos_MeshInventory suitable for the Entities we want to attach them too to test spawn, orientate, them on the character
6. Build the Logic in PROTOS_EntityComponent to achieve basic ingame Arbitrary Attachment. Add GUI as need to alter data values.
7. [R&D] Build the Logic in PROTOS_EntityComponent to devise Specified Attachment and Rules based filtering. Add GUI as need to alter data values.
8. Iterate and expand functionality

PROTOS

Blueprints

Actor Component

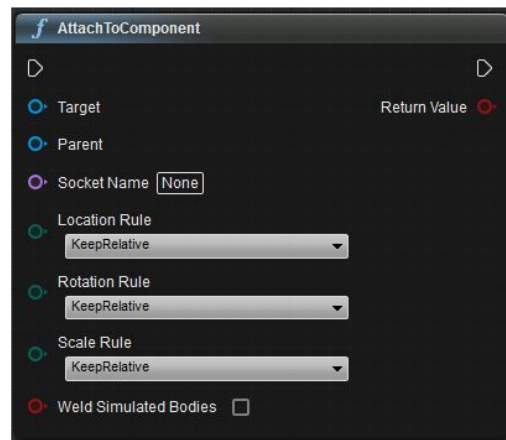
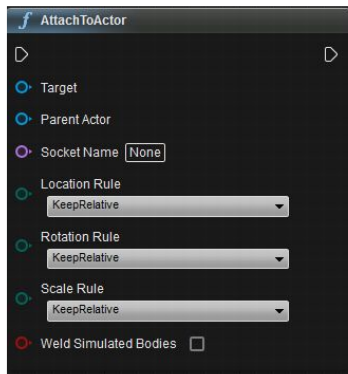
Spawn Actor

Attach To Actor

[Add Components](#)

[Attach Components](#)

[Transformation](#)



P R O T O S

Saving /Loading Configurations (JSON)

Mesh parts are attached to the skeletal bones/sockets or root pivot, after the PROTOS configuration has been retrieved. The Mesh Part themselves can have their own configurations with info for attaching smaller mesh pieces.

P R O T O S

Performance

Static Mesh Layers

Material Instancing

Level Of Detail

Custom Occlusion

PROTOS

GOLEMCRAFT
Humanoid/Creatures

MONSTAMASH

GLAMOUR
Garments/Armour

UMECHS
Vehicle/Machines

GUNVERTOR
Fire Weapon

GLAIVERAVE
Melee Weapons

HIERARCH
Architecture
Furniture

YAMAIA
Land/Oceanscape

BOTANNY
Plants'/Foliage

MAUI
GUI Panel

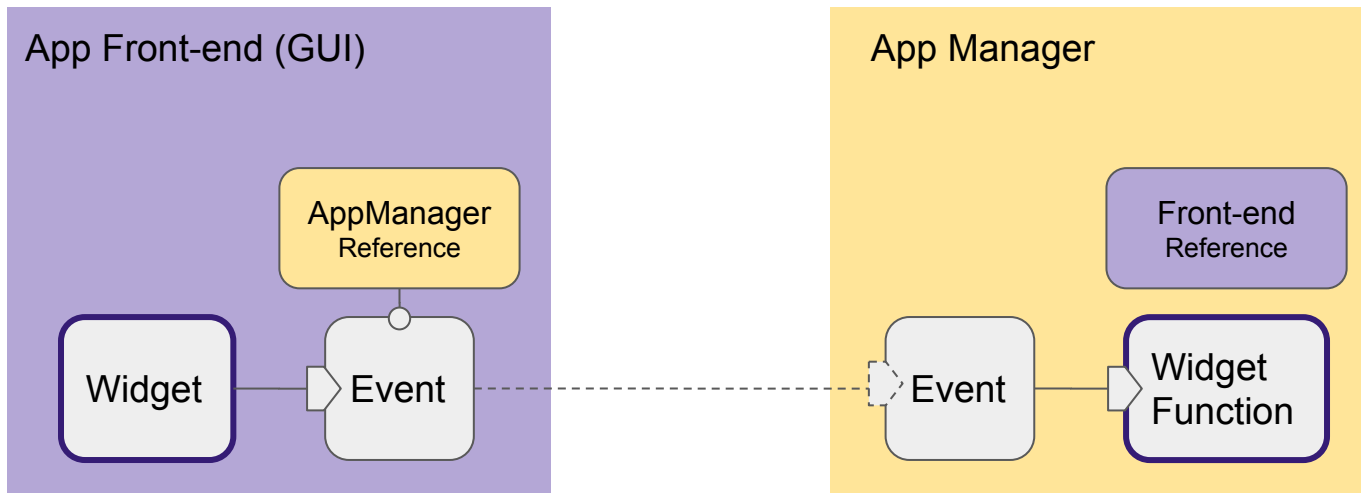
SongCubes
Visual (3D Cube)
3D Soundque Sequencer

MOODSTER
Dynamic Music
System

FIRECORE
Dynamic Weapon's
SFX System

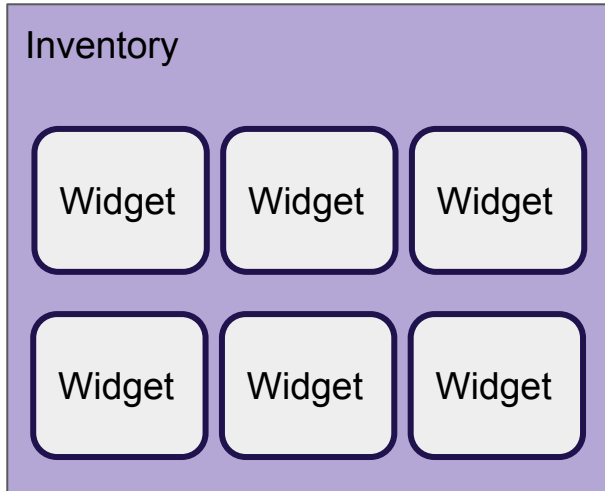
PROTOS

Decoupled GUI



PROTOS

GUI Inventory



Item Drag n Drop + Click to Select

Photography/Iconography

Movable GUI Windows

PROTOS

Controller

- Viewport
- Camera Gizmo → Pan/Tilt/Zoom/ Orbit
- 3D Object Gizmo → Position/Rotate/Scale/Select w/ Mouse | Gamepad
- 3D Pointer Gizmo
- 3D Area Selector (Cubic)

Objects

World
Entity

BaseBody
Skeletons
Sockets

Spawner Points/Volumes: Mesh, Instanced Static Mesh Layer, Procedural Mesh, Particle Systems
Symmetric Spawner - associates a part with another, will spawn the parts attached to primary w mirror/flip/invert/enable|disable options.
Allows arbitrary symmetric part generation.

Waypoints - used to create automated paths and teleport points
Bot Spawner
Projectile Spawner -

Toolbox

Collage Construction

Exterior Spawners

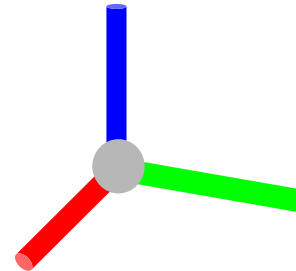
Particle Paint Emitters

Garbage Paint (Randomized Fragments)
Emitter

Clusters (Pattern Array)

Interior Spawners

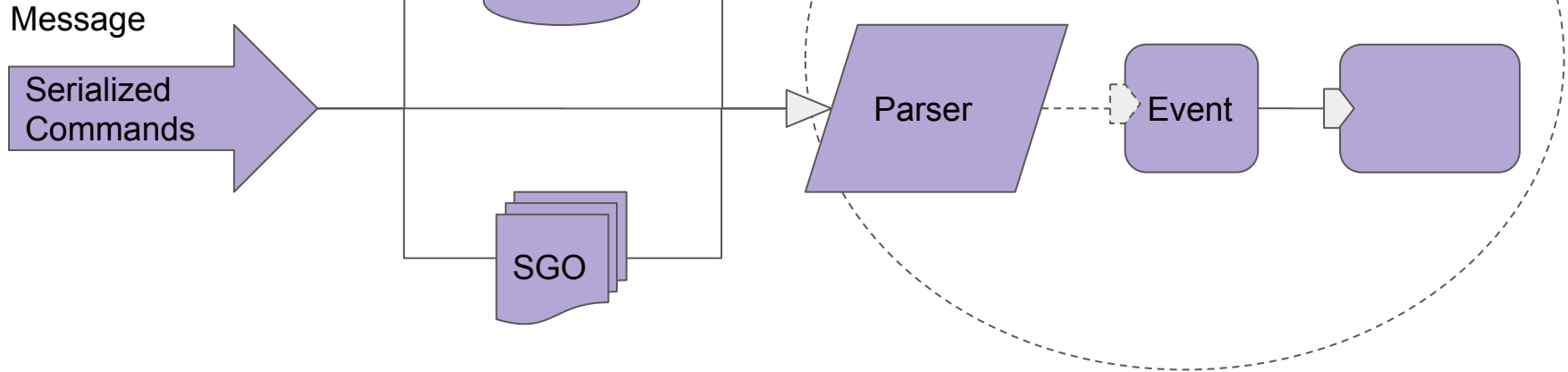
Socket-based (Hard,Soft,Smart) Attachment
Symmetric Spawn & Attachment
Individual & Group Piece (Fragment) Selection, + Control Orientation
Snap on: Location, Rotation, Scale, Lock Axis on Adjust
Snap to: Bone/Socket
Create Swap List/Swap Part



PROTOS

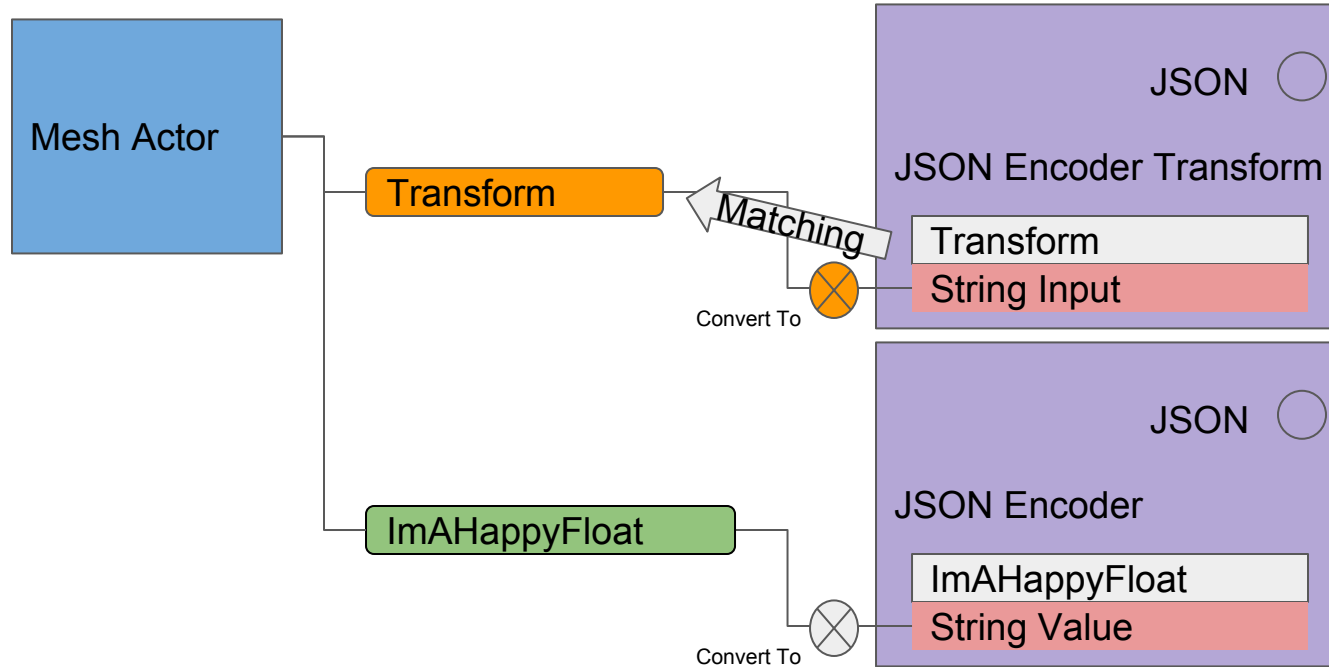
Serialized Communications
For Internal and External Use

Human Readable (JSON)



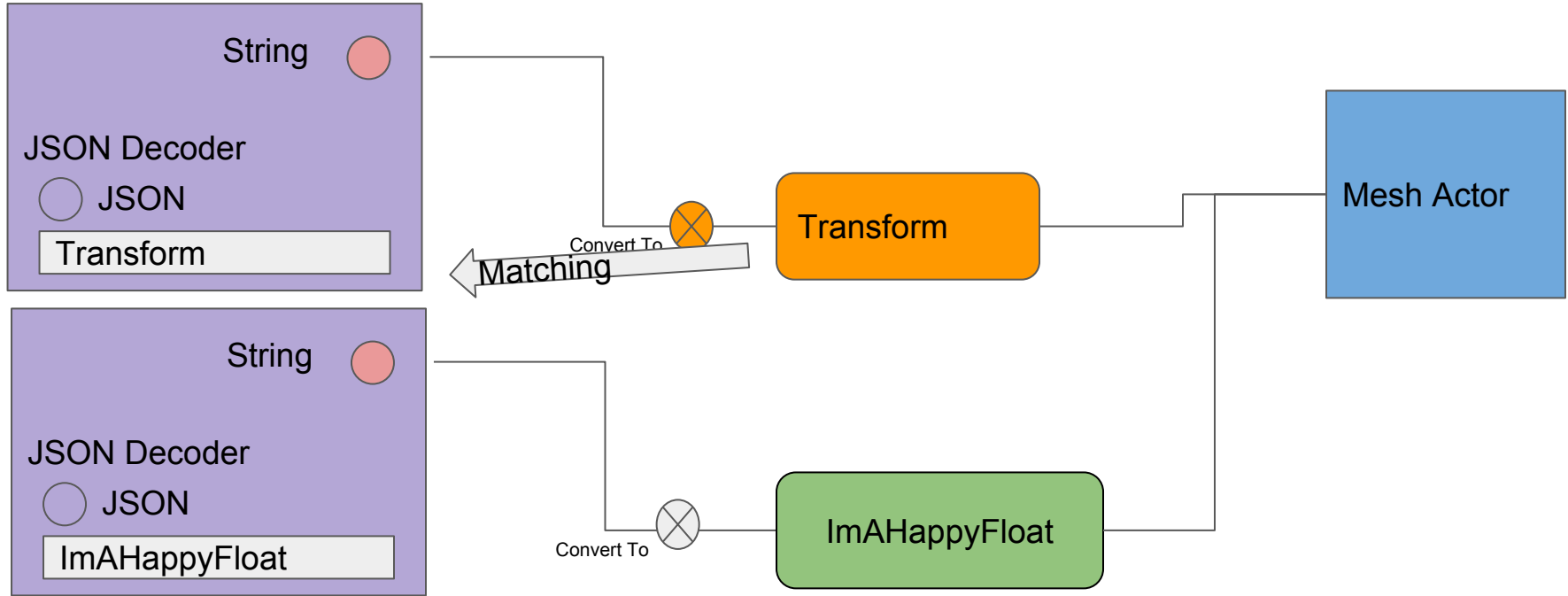
PROTOS

JSON Encoder



PROTOS

JSON Decoder

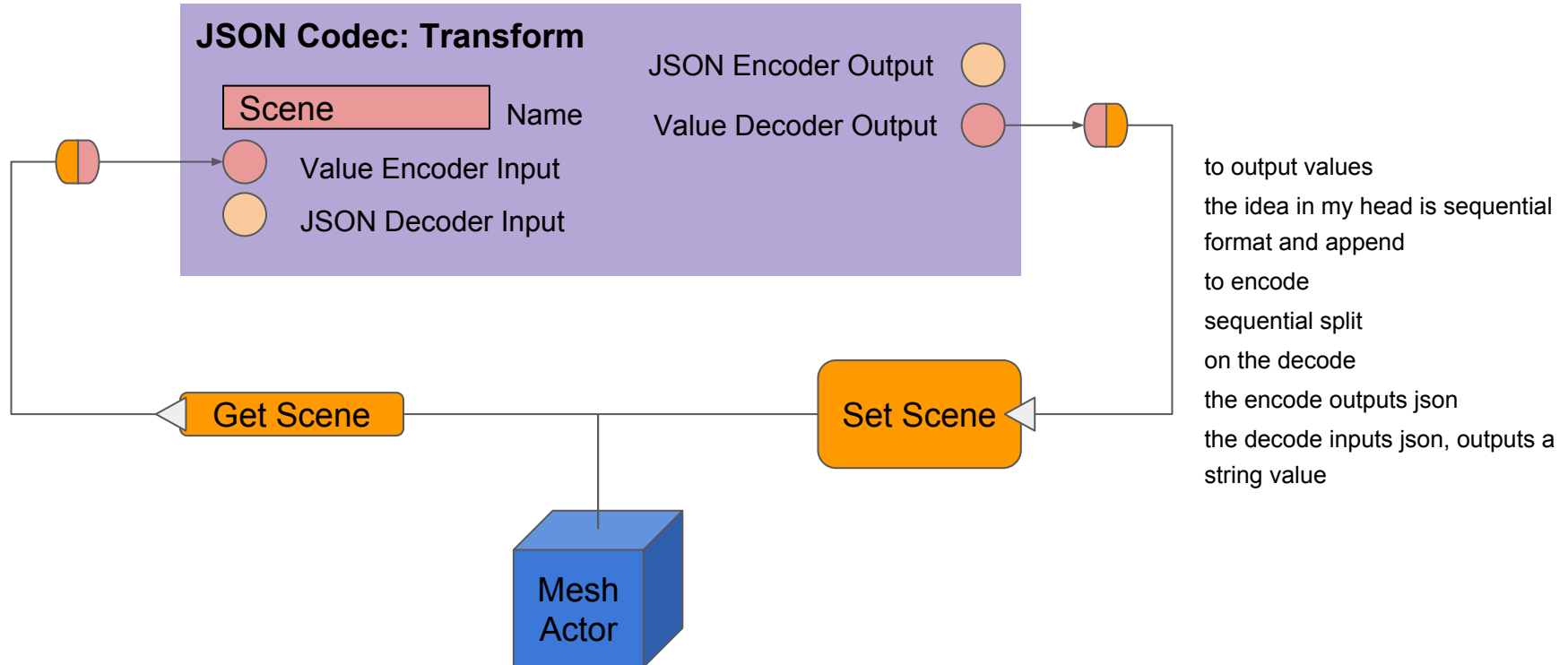


PROTOS



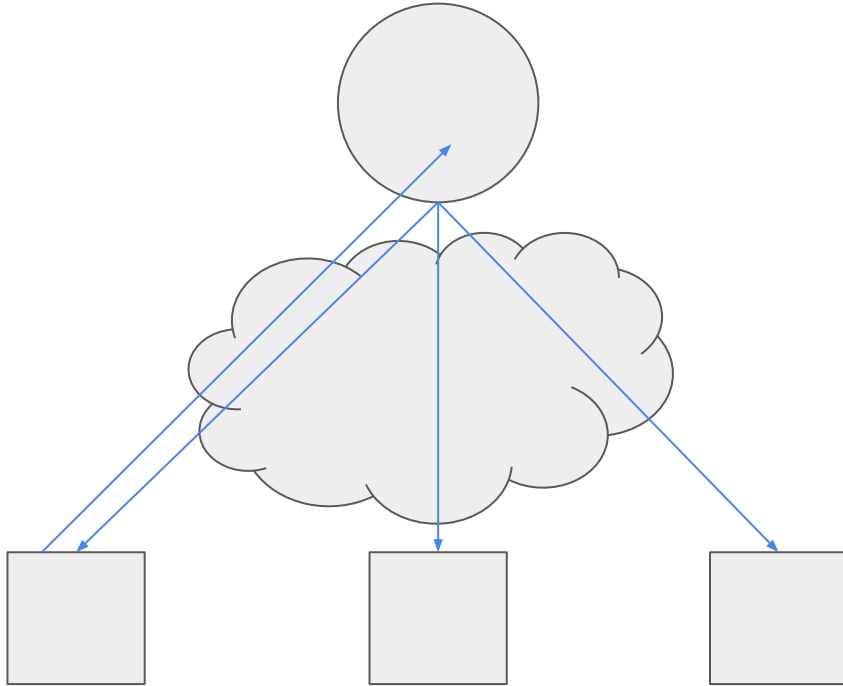
JSON Codec

Combined Encoder/Decoder()





Network Transfer and Storage System



PROTOS

Resource: { Short-term:{Editor|Cache}, Long-term:{Slot|HTTPServer} }

UHost: {UServer,UClient}

Recipient: {Specific UClient,Group UClients,All UClients,HTTPServer, HTTPServer Group}

Send [Resource] From [UHost] to [Recipient] via UServer

Request [Resource] from [UHost]

A Request from is message from Recipient to Host to Send

Save [Short|Long-term] to [Short|Long-term]

Load [Short|Long-term] to [Short|Long-term]

Send + Save

Send + Save + Load

Request + Save

Observer Pattern: Publish/Subscribe to create groups on UServer